

Unit 1: ReactJS Basics

Introduction to ReactJS:

- React is a front-end JavaScript library.
- React was developed by the Facebook Software Engineer Jordan Walke.
- React is also known as Reactjs or ReactJS.
- React is a tool for building UI components.
- React is a JavaScript library for building user interface.
- React is used to build single page Application. (It's used for building **Single Page Applications (SPAs)** where content updates without reloading the whole page).
- React allows us to create reusable UI components.

```
function Button(props) {  
  return <button>{props.label}</button>;  
}
```

// Reusing the component

```
<Button label="Submit" />
```

```
<Button label="Cancel" />
```

```
<Button label="Save" />
```

Each Button above is the same component, just reused with different data (label prop). That's the power of reusable UI components in React.

- React is free and open-source library.

Free to use: Anyone can download, use, and modify it without any cost.

Open-source: Its source code is publicly available on [GitHub](#)

GitHub is a **platform for hosting and collaborating on code**, especially for projects that use **Git** for version control.

- React is a declarative, efficient and flexible JavaScript library for building user interface. It lets you compose complex UIs from small and isolated pieces of code called "components".
(In programming, **declarative** means you describe **what** you want to happen, not **how** it happens step-by-step.)
- React community also provides advanced concept like state management, routing etc., on top of the react library.

How does React Work?

- React create VIRTUAL DOM in memory. (DOM-Document Object Model).
- Instead of manipulating the browser's DOM directly, React creates a virtual DOM in memory, where it does all the necessary manipulating, before making the changes in the browser DOM.

Browser DOM

React Virtual DOM

Applications: –

Few Popular websites powered by react library are listed below–

- Facebook, popular social media application.
- Instagram, popular photo sharing application.
- Netflix, popular media streaming application.
- Reddit, popular content sharing application.

Single Page Applications (SPAs)

- React is ideal for SPAs, where only one HTML page loads and different views render dynamically without refreshing the entire page.
- **Example:** Gmail, Instagram.

Dynamic Web Applications

- Useful for applications where the content or layout changes based on user interaction or live data.
- **Example:** Weather apps, live search filters.

Mobile Applications (Using React Native)

- React's component logic can be used in React Native to create cross-platform mobile apps for Android and iOS.
- **Example:** Facebook, Instagram mobile apps.

E-commerce Platforms

- React is great for building interactive product listings, real-time cart updates, and filters in e-commerce websites.
- **Example:** Flipkart (some parts), Shopify storefronts.

Real-Time Data Applications

- React can render updates in real-time using APIs or WebSockets without refreshing the page.
- **Example:** Stock market dashboards, sports live scores.

Social Media Platforms

- React handles complex user interfaces like feeds, likes, comments, and notifications effectively.
- **Example:** Facebook uses React in its frontend.

Content Management System (CMS) Frontends

- React is used as a frontend with headless CMS (like WordPress or Strapi) for fast and customizable content display.
- **Example:** Blogs or news portals using React + Headless CMS.

SaaS (Software as a Service) Dashboards

- React helps create interactive user dashboards for SaaS tools like CRMs, analytics, or project management systems.
- **Example:** Jira-like interfaces, CRM tools.

Progressive Web Apps (PWAs)

- React supports building PWAs which can work offline, load fast, and provide app-like features in a browser.
- **Example:** Twitter Lite, Pinterest Lite.

Learning/Educational Platforms

- Great for building interactive quizzes, video content, progress tracking, and dynamic course content.
- **Example:** Online platforms like Udemy, Khan Academy (conceptually).

Portfolio and Resume Websites

- Developers use React to build their personal portfolios with animations, project lists, and contact forms.
- **Example:** Interactive resume websites.

Blog Frontends Using Headless CMS

- Blogs can be developed using React to fetch content from APIs and render it efficiently.
- **Example:** Developer blogs using Gatsby or Next.js.

Streaming Platforms (Video/Audio)

- React helps in creating responsive video players and managing real-time streams.
- **Example:** Netflix-like UI (projects or clones).

Chat and Messaging Applications

- Real-time messaging UIs benefit from React's fast re-rendering.
- **Example:** Slack/WhatsApp clone projects.

Admin Panels and Control Dashboards

- Useful for managing users, settings, and data in a visual and real-time way.
- **Example:** Admin dashboards for internal company tools.

Survey or Form Builder Applications

- React handles complex, multi-step, and dynamic form elements smoothly.
- **Example:** Google Forms clone.

Job Portals and Recruitment Platforms

- React is used to build interactive job boards, search filters, and application tracking.
- **Example:** LinkedIn clone projects.

Travel Booking Interfaces

- React can power real-time search, filtering, booking, and calendar integration.
- **Example:** MakeMyTrip UI (project-based).

Healthcare Portals

- React helps build patient dashboards, appointment systems, and health record viewers.
- **Example:** Doctor-patient appointment apps.

News and Weather Apps

- Ideal for updating live headlines, weather forecasts, and user-personalized news feeds.
- **Example:** News API + React-based frontend.

Advantages of Using Reactjs:

Reactjs is a powerful JavaScript library for building dynamic and high-performance user interfaces.

1. Reusable components:

- React promotes a component-based architecture, where developers can break down the UI into small, self-contained units of code, known as components.
- These components are reusable, making the development process more efficient and the codebase cleaner.
 - Improved maintainability: Isolated components make updates or bug fixes easier.
 - Faster development: Reusable components save time and effort on repeated elements.

2. Efficient Rendering with Virtual DOM:

- React uses a Virtual DOM to improve the performance of web applications. Instead of updating the real DOM directly, React first updates the Virtual DOM and compares it to the previous one, applying only the minimal changes required
- Faster updates: Updates only the changed parts of the UI.
- Improved performance: Reduces the number of expensive DOM manipulations.

3. SEO-Friendly:

- React offers features that help make applications SEO-friendly, including Server-Side Rendering (SSR) and Static Site Generation (SSG).
- These methods allow React to generate content that search engines can easily crawl and index.
- Better search rankings: Ensures content is indexed by search engines.
- Faster page loads: Improves user experience and SEO by reducing load times.

(When building modern web applications, Search Engine Optimization (SEO) plays a critical role in ensuring your site ranks well on search engines like Google. But one common challenge when using JavaScript libraries like React is that search engines often struggle to properly crawl and index content that's generated on the client-side.)

4. Fast Rendering

- React ensures fast rendering by utilizing the Virtual DOM and the Reconciliation Algorithm. These mechanisms minimize unnecessary re-renders, improving the responsiveness of the application.

5. Cross-Platform Mobile Development with React Native:

- React Native allows developers to use React to build native mobile apps for iOS and Android with a single codebase. This leads to faster development and code reuse across platforms.
- Single codebase: Develop apps for both iOS and Android.
- Native performance: Offers near-native app performance.

6. Redux for State Management:

- Managing application state can become complex, especially in large apps. Redux helps manage state in a predictable way by maintaining the entire state in a single store.
- Centralized state: Easier tracking and management of the application's state.
- Predictable state updates: Changes can only happen through dispatched actions.

7. JSX – JavaScript Syntax Extension:

- JSX is a syntax extension for JavaScript, allowing developers to write HTML-like code within JavaScript. This improves the readability and structure of the code, making React development more intuitive.
- Improved readability: Combines JavaScript logic and HTML structure.
- Better code organization: UI structure is closely tied to components.

8. Flexibility and Performance Optimization:

- React offers flexibility with its component-based architecture and supports different rendering methods. Its performance is optimized using features like `React.memo`, code-splitting, and lazy loading.
- Flexible integration: Easily integrates with other tools and frameworks.
- Optimized performance: Features like code-splitting and lazy loading reduce load times.

9. Strong Community Support

React.js has a large and active community backed by Meta (formerly Facebook), providing an extensive ecosystem of libraries, tools, and resources for developers.

- **Vast resources:** Plenty of documentation, tutorials, and third-party libraries.
- **Active community:** Easy access to solutions and support through forums and GitHub

Virtual DOM: –

- The Virtual DOM (VDOM) is an in-memory representation of the real DOM elements of a web page.
- Unlike the actual DOM, which is a tree-like structure that the browser uses to render content on the screen, the Virtual DOM is a lightweight, JavaScript-based abstraction that exists entirely in memory.
- In simpler terms, the Virtual DOM is a copy of the real DOM that React maintains internally. By manipulating this virtual representation rather than the actual DOM, React can perform updates more efficiently.

The **Virtual DOM (VDOM)** is a key feature that helps React build **fast and efficient user interfaces**. It allows React to **update the UI without reloading the whole page or touching every element**.

◆ Step-by-Step: How Virtual DOM Works

1. Initial Render:

- You write JSX (e.g., `<h1>Hello</h1>`)

- React **creates a Virtual DOM** (a JS object that mimics the real DOM)
- This VDOM is then **used to update the actual DOM** (rendered on screen)

2. UI Update Triggered:

- A user does something (clicks a button, types text)
- This changes the component's **state** or **props**

3. New Virtual DOM Created:

- React **re-renders** the component and creates a **new Virtual DOM tree**
- It **doesn't immediately update** the real DOM

4. Diffing (Comparison):

- React **compares the new VDOM** with the **old VDOM**
- This process is called **“diffing”**
- It finds out **what has changed** (which nodes, text, attributes, etc.)

5. Patching (Update Real DOM):

- React **updates only the changed parts** in the real DOM
- This is done using **minimal changes**, avoiding full reloads



Example

Let's say the DOM looks like:

```
jsx
```

CopyEdit

```
<h1>Hello</h1>
```

Then you change the text to:

```
jsx
```

CopyEdit

<h1>Hello, World!</h1>

- React creates a **new VDOM**: <h1>Hello, World!</h1>
- It compares with the **old VDOM**: <h1>Hello</h1>
- It finds that only the **text changed**
- Only that part is updated in the real DOM (not the entire element)

Setting up a development Environment: –

1) Install visual studio code

2) Add Extension: –

- **Babel JavaScript**– It assign the popper color code to reserve keyword; it also provides IntelliSense support according to ReactJS.

Babel is a JavaScript compiler that allows you to write modern JavaScript (ES6 and beyond) and convert (transpile) it into older JavaScript that all browsers can understand.

Babel takes new JavaScript code (like ES6+) and converts it into backward-compatible JavaScript (like ES5) so it runs on older browsers.

- **JavaScript es6 code snippets**: –

ReactJS use modern JavaScript es6–

helps you write React and JavaScript code faster using shortcuts (snippets)

provides shorthand snippets to insert common JavaScript and React code blocks instantly.

- **vscode icons**: proper icons to the ReactJS file.

- **Bracket highlighter**:

It helps you visually match **brackets, parentheses, curly braces**, and **quotes** in your code by **highlighting** the matching pairs.

- **Auto rename tag**

3) Install Node JS & NPM: –

React require s Node.js so begin by installing the latest version of Node.js from the official Node.js website(<http://node.org>). node.js comes with NPM (Node Package Manager) which is used to manage project dependencies.

Check the version :-

C:/> node -v

C:/> npm -v

4)Installing ReactJS from VS Code Terminal

Installing ReactJS:-

Step 1: Install VS Code&Extension

Step 2: Install Node JS&NPM

After installing Node JS&NPM check the version of the node JS&NPM.

Open VS Code Terminal or Command prompt and type following command-

Check the version :-

C:/> node -v

C:/> npm -v

Step 3: Create a folder in any drive ex- ReactJS

Step 4: Installing ReactJS from VS Code Terminal

```
npm install -g create-react-app
```

:- It create basic directory structure and is used to **globally install** the **create-react-app** package on your system using **npm (Node Package Manager)**.

npm: Node Package Manager, used to install JavaScript packages.

install: The action you want to perform— install a package.

-g: Stands for global. It installs the package system-wide, so you can use it from any directory via the terminal/command prompt.

create-react-app: The package that sets up a new React application with a pre-configured build system (Webpack, Babel, etc.).

Note (as of recent versions):

The **recommended** way now is to use npx instead of installing create-react-app globally:

```
npx create-react-app my-app
```

This **doesn't require global installation**, and it **always uses the latest version** of create-react-app

Command Purpose

npm install -g create-react-app Installs create-react-app globally

npx create-react-app my-app
Runs the latest version without installing globally

Check the version of “create- Using

create-react-app -- version

Creating React App by using

create-react-app “App_Name“

Ex- create-react-app new-app

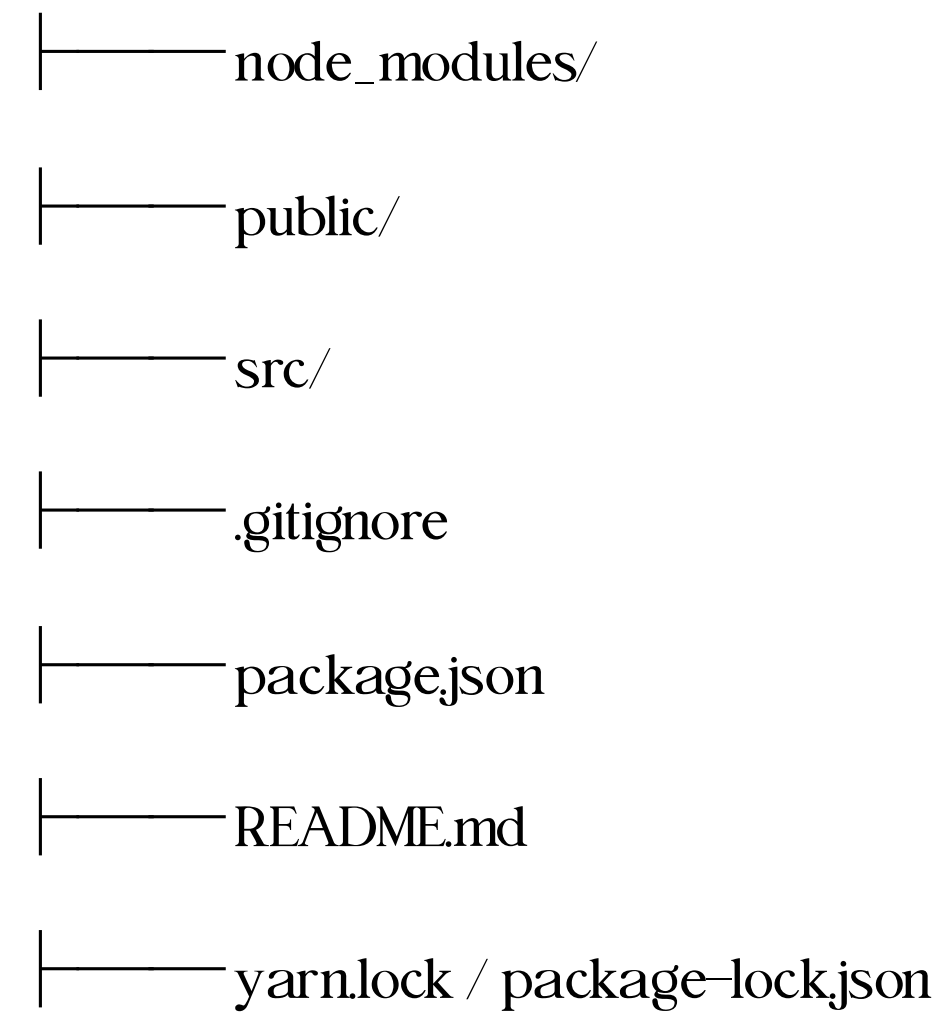
cd new-app

npm start

Here's a simple explanation of the **default file structure of a React app** created using create-react-app (CRA), such as when you run:

npx create-react-app my-app

my-app/



Now let's break it down:

✓ 1. node_modules/

- Contains all installed npm packages your app depends on.
- You usually don't touch this directly.

✓ 2. public/

Used to store static files like HTML, images, icons, etc.

Important files:

- **index.html:**
The single HTML file used by React. This is where your entire React app is injected via `<div id="root">×</div>`.
- **favicon.ico:**
Browser tab icon.
- **manifest.json:**
Used for Progressive Web Apps (PWA).

⚠ Note: Don't put JS/React files here. React works from src/.

✓ 3. src/

Where all your React code lives (JS, JSX, CSS, components, etc.)

Important files:

- **index.js:**
Entry point of the app. It renders the top-level `<App />` component into `public/index.html`.
- **App.js:**
Main app component. You write your UI logic here or break it into smaller components.
- **App.css:**
Styling for App.js.
- **index.css:**
Global styles.
- **logo.svg**
Sample logo used in App.js.

You can add folders like `components/`, `pages/`, `utils/`, etc., inside `src/` as your app grows.

4. `.gitignore`

Lists files/folders Git should ignore (like `node_modules`, `build`, etc.)

5. `package.json`

Very important!

It includes:

- App name and version.
- Dependencies (react, react-dom, etc.)
- Scripts (npm start, npm run build, etc.)
- Configuration for the project.

6. `README.md`

Markdown file with basic instructions and project description.

7. `yarn.lock` or `package-lock.json`

Lock file for dependency versions — ensures consistent installs.

Summary of Flow:

1. Browser loads `public/index.html`.
2. React mounts `<App />` from `src/index.js` into `<div id="root">`.
3. You build your app inside `App.js` and other files in `src/`.

Starting React On Server:

`npm start` :- it serves our recat app on 3000 port number of localhost.

Understanding JSX: –

JSX stands for JavaScript XML, and it is a special syntax used in React to simplify building user interfaces. JSX allows you to write HTML-like code directly inside JavaScript, enabling you to create UI components more efficiently. Although JSX looks like regular HTML, it's actually a syntax extension for JavaScript.

JSX combines **HTML** and **JavaScript** in a single syntax, allowing you to create **UI components** in React. It simplifies rendering dynamic content by embedding **JavaScript** expressions inside **HTML-like tags**.

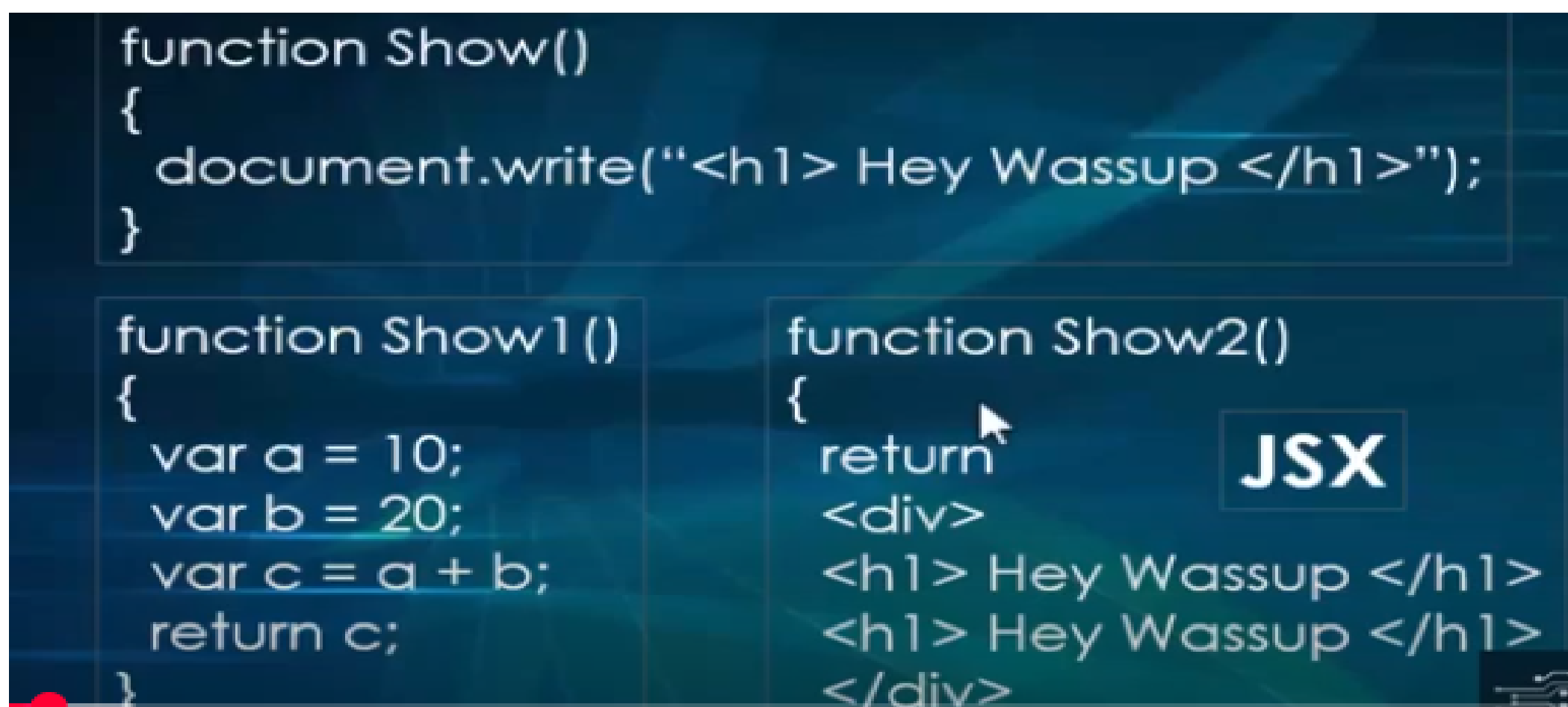
JSX stands for **JavaScript XML**.

JSX is an extension of javascript language

JSX allows us to write HTML in react

JSX makes it easier to write and add HTML in react

It allows you to write **HTML-like code** inside JavaScript — used heavily in React.



Why JSX?

React components return **UI**. JSX helps write this UI in a way that feels like HTML but is actually JavaScript under the hood.

```
function MyComponent() {

  return <p>This is JSX!</p>;

}
```

The above code gets compiled into:

```
React.createElement('p', null, 'This is JSX!');
```

Note:– `React.createElement()` is a **core method** used by React to **create virtual DOM elements**.